

Information Theory and Data Compression - 2026

Lecture 1 [Introduction + LZ77]

Dr. Asmaa Ahmed E. Osman

Faculty of Computers and Artificial Intelligence

Course Reference

“Introduction to Data Compression”

By Khalid Sayood

Fourth Edition , 2012

(The Morgan Kaufmann Series in Multimedia
Information and Systems)

*slides adapted from Dr. Khaled Mostafa slides

Course Logistics

Staff:

Dr. Asmaa Ahmed, E-mail: asmaa.a.elsayed@fci-cu.edu.eg

Classroom code: rzix5dwp

Grading

- Midterm → 20%
- Labs + Quizzes → 20 %
- Final → 60%

Student Course Coordinator !!



Data Compression

“**Data Compression** is the art or science of representing information in a compact form”

Data Compression

- What is **Multimedia**?

Derived from the word “**Multi**” and “**Media**”

- Multi - Many, Multiple
- Media - Distribution tool & information presentation

(text, graphic, voice, images, music and etc.)

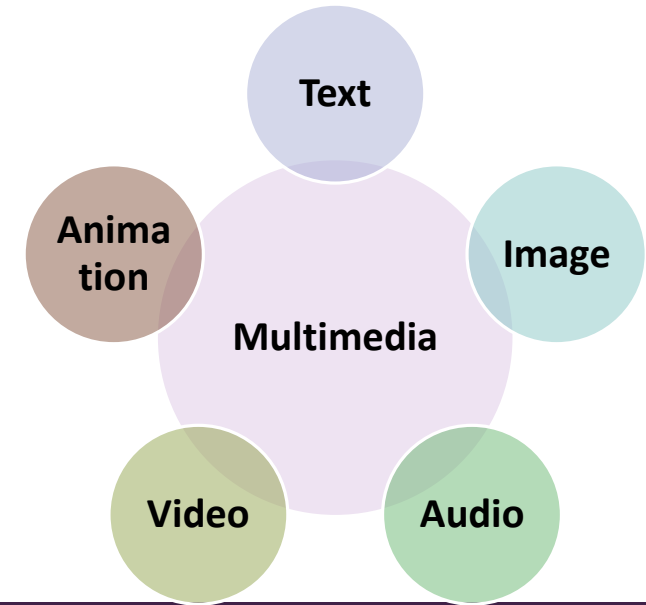


Image Data Size

Gray Image (one Byte / Pixel)

For 1024*768 Pixel Gray Image

Original Size = $1024 * 768 * 1 \text{ Byte} = 768 \text{ K bytes}$

Color Image (Three Bytes / Pixel {Red, Green, Blue})

For 1024*768 Pixel Color Image

Original Size = $1024 * 768 * 3 \text{ Bytes} = 2304 \text{ K bytes}$

Video Data Size

Video (25 Frame / Second)

For 1 Minute 1024*768 Pixel Video clip

- Original Size (for 1 Sec) = $1024 * 768 * 3 \text{ Bytes} * 25 \text{ Frames} = 57600 \text{ K bytes}$
- Original Size (for 1 Min) = $1024 * 768 * 3 \text{ Bytes} * 25 \text{ Frames} / \text{Sec} * 60 \text{ Sec/Min} = 57600 * 60 = 3456000 \text{ K bytes} = 3.456 \text{ GB}$
- What About 2 Hours Movie ?? ($3.456 * 120 \text{ Min} = \text{!!!!}$)
- What of using NTSC system (30 Frame / Sec) !!!

Why Compress?

- To reduce the **volume of data to be transmitted** (text, fax, images)
- To reduce **storage requirements** (speech, audio, video)
- To reduce the **bandwidth required for transmission**

How is compression possible?

- Redundancy in digital audio, image, and video data
- Properties of human perception

Digital audio is a series of sample values;

Image is a rectangular array of pixel values;

Video is a sequence of images played out at a certain rate

Neighboring sample values are correlated

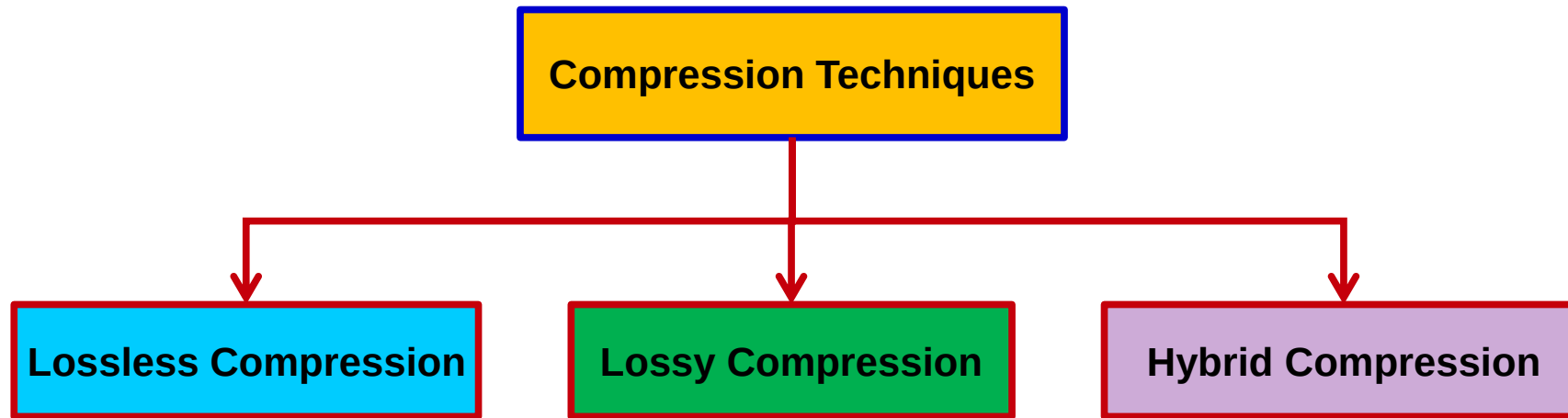
(1) Redundancy

- Adjacent audio samples are similar (predictive encoding); samples corresponding to silence (silence removal)
- In digital image, neighboring samples on a scanning line are normally similar (spatial redundancy)
- In digital video, in addition to spatial redundancy, neighboring images in a video sequence may be similar (temporal redundancy)

(2) Human Perception Factors

- Compressed version of digital audio, image, video need not to represent the original information exactly
- ❑ Perception sensitivities are different for different signal patterns
- ❑ Human eye is less sensitive to the higher spatial frequency components than the lower frequencies

Covered Compression Techniques



Classification of Compression Techniques

[1] Lossless compression

lossless compression for legal and medical documents, computer programs “*exploit only data redundancy*”

[2] Lossy compression

digital audio, image, video where some errors or loss can be tolerated “*exploit both data redundancy and human perception properties*”

[3] Near Lossless Compression

It is a lossy compression with a predefined max accepted error

[4] Hybrid Techniques

A compression algorithm that utilizes many **lossy/lossless** techniques to achieve high compression ratio with best quality. (.e.g. JPEG, MPEG, H264,..)

Image Quality

Gray Image 400 * 500 Pixels

Image Size = $400 * 500 * 1 \text{ byte/pixel}$
= 200,000 byte = ~ 200 Kbyte

What will be the degradation in Quality if this image is compressed using lossy compression ?

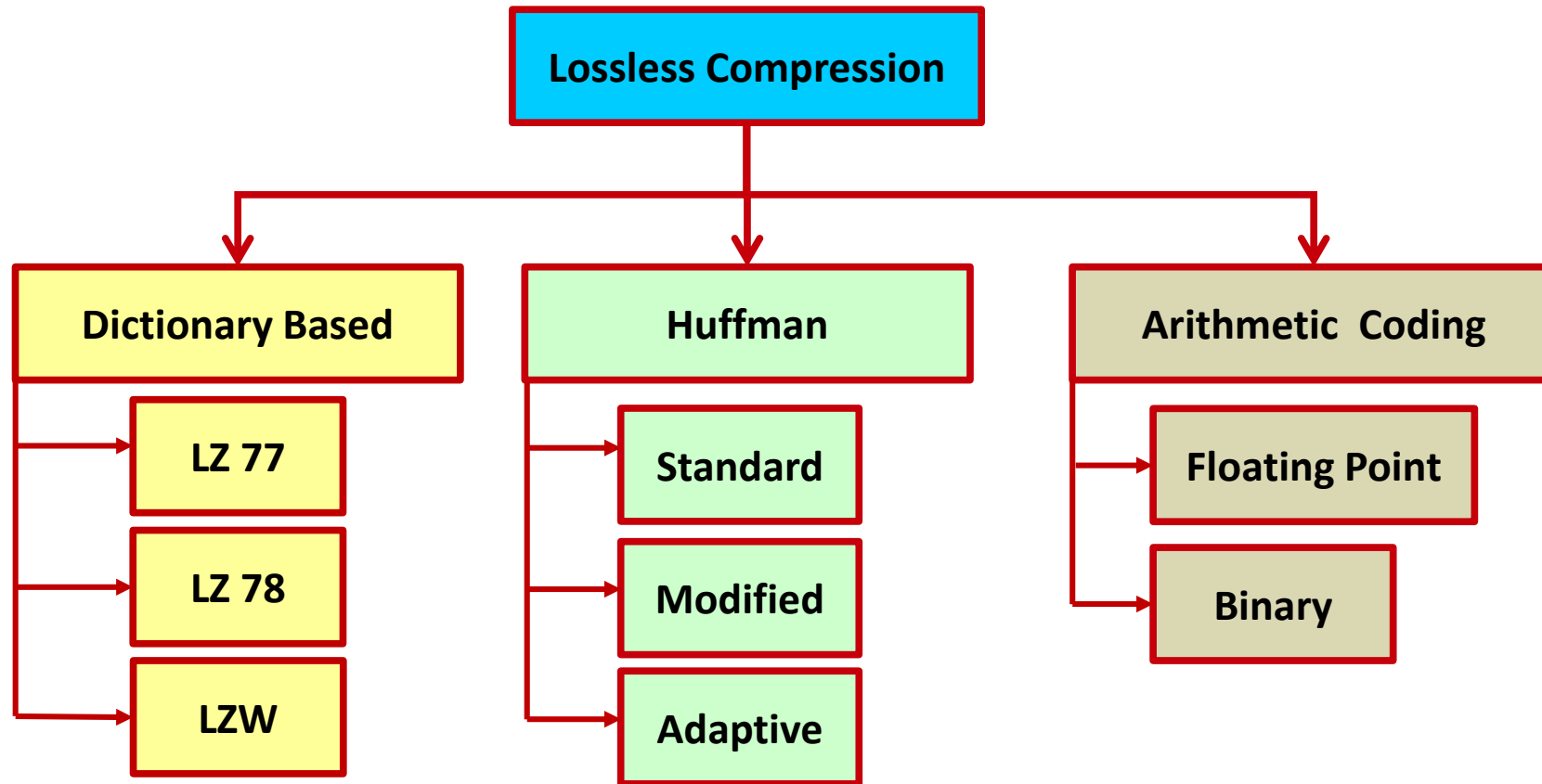


Sorry, this is the compressed version with size 38Kbyte Only
Is this quality accepted for you ??
The compressed size is about 1/5 of the original size

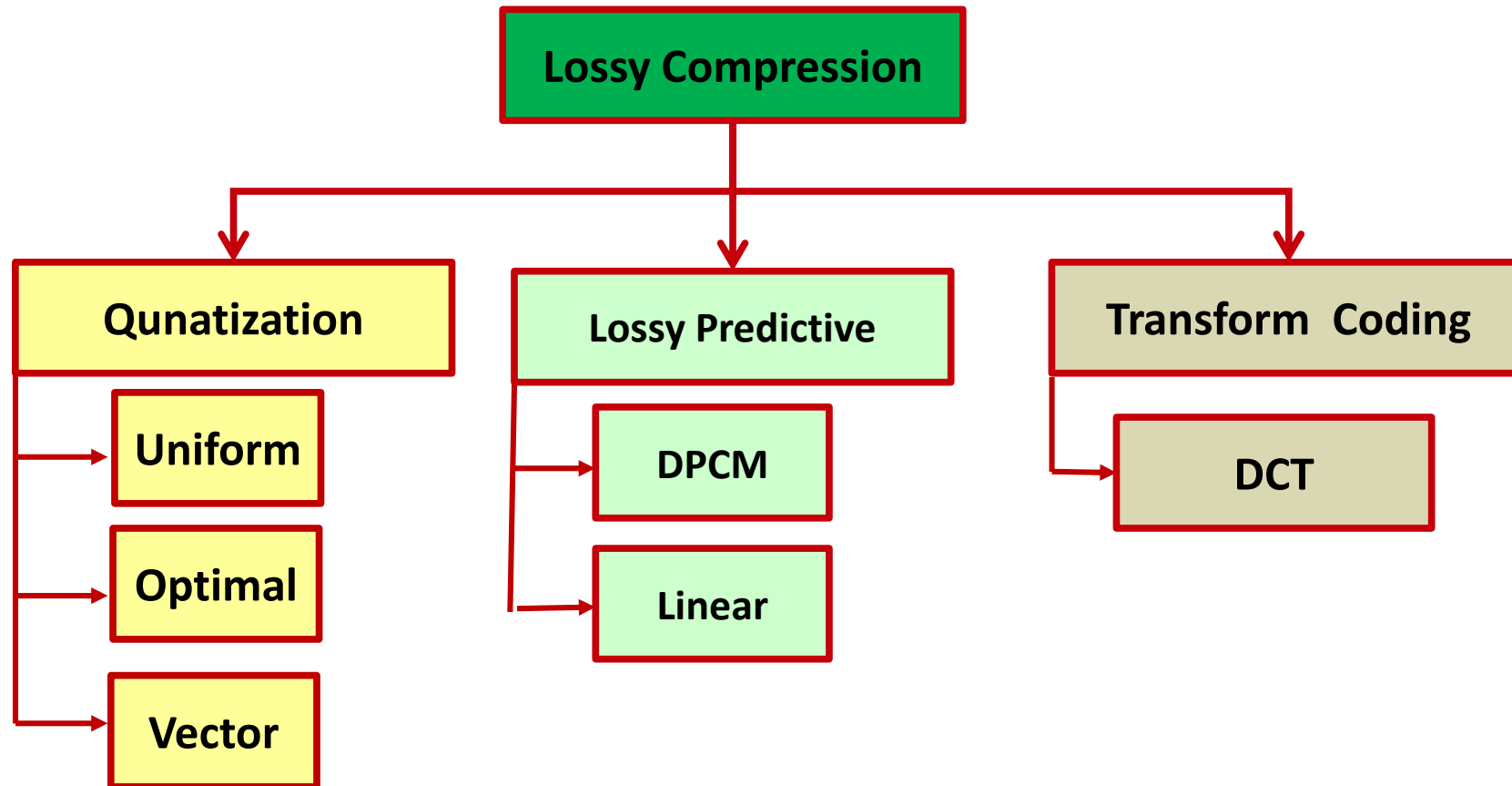
All These Images are Lossy Compressed images



Covered Compression Techniques



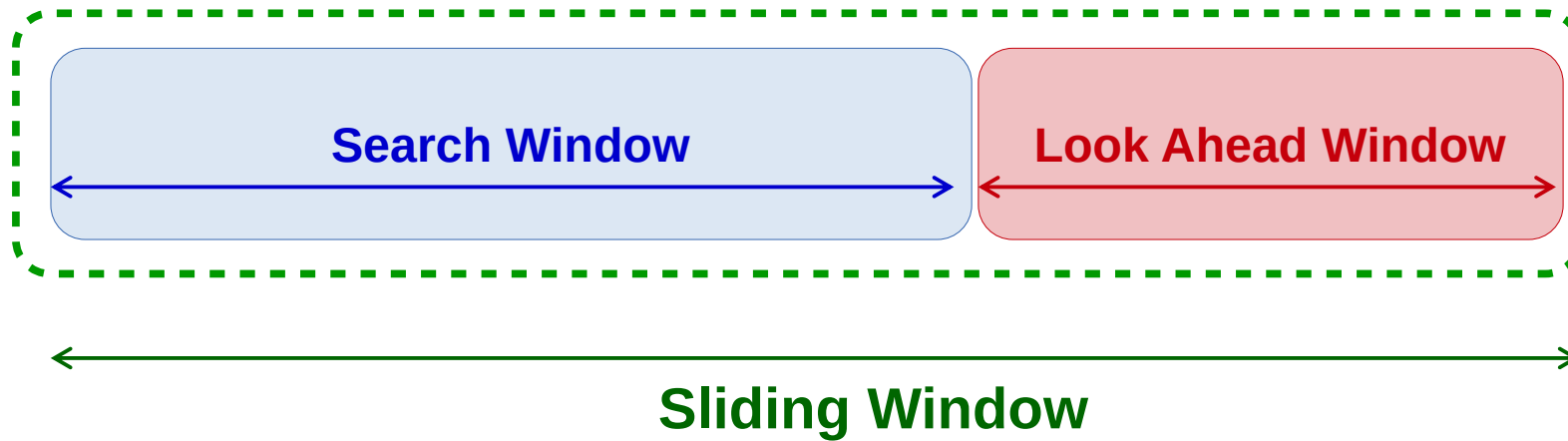
Covered Compression Techniques



Dictionary Based Compression

- These techniques—both static and adaptive (or dynamic)—build a list of commonly occurring patterns and encode these patterns by transmitting their index in the list
- They are most useful with sources that generate a relatively small number of patterns quite frequently, such as text sources
- A very reasonable approach to encoding such sources is to keep a list, or **dictionary**, of frequently occurring patterns
 - When these patterns appear in the source output, they are encoded with a reference to the dictionary
 - If the pattern does not appear in the dictionary, then it can be encoded using some other less efficient method (we lose in this case)

Lempel Ziv 77 (LZ 77) Algorithm



A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

TAG ➡ **<Position , Length , Next Symbol >**

LZ 77 Algorithm

Search Buffer: It contains a portion of the recently encoded sequence.

Look-Ahead Buffer: It contains the next portion of the sequence to be encoded.

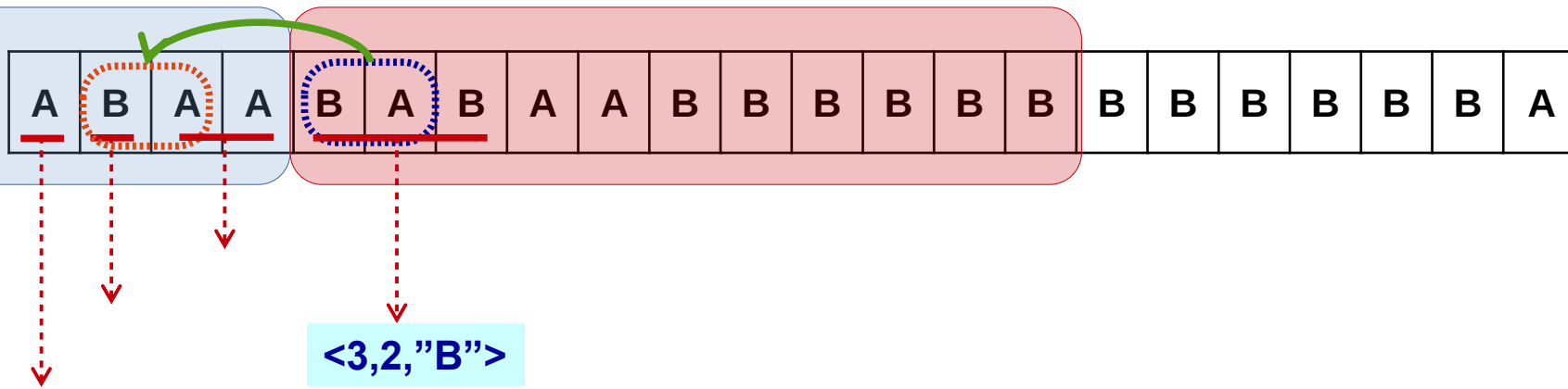
Once the longest match has been found, the encoder encodes it with a triple
<Position , Length, Next Symbol >

Position : the offset or position of the longest match from the lookahead buffer

Length : the length of the longest matching string

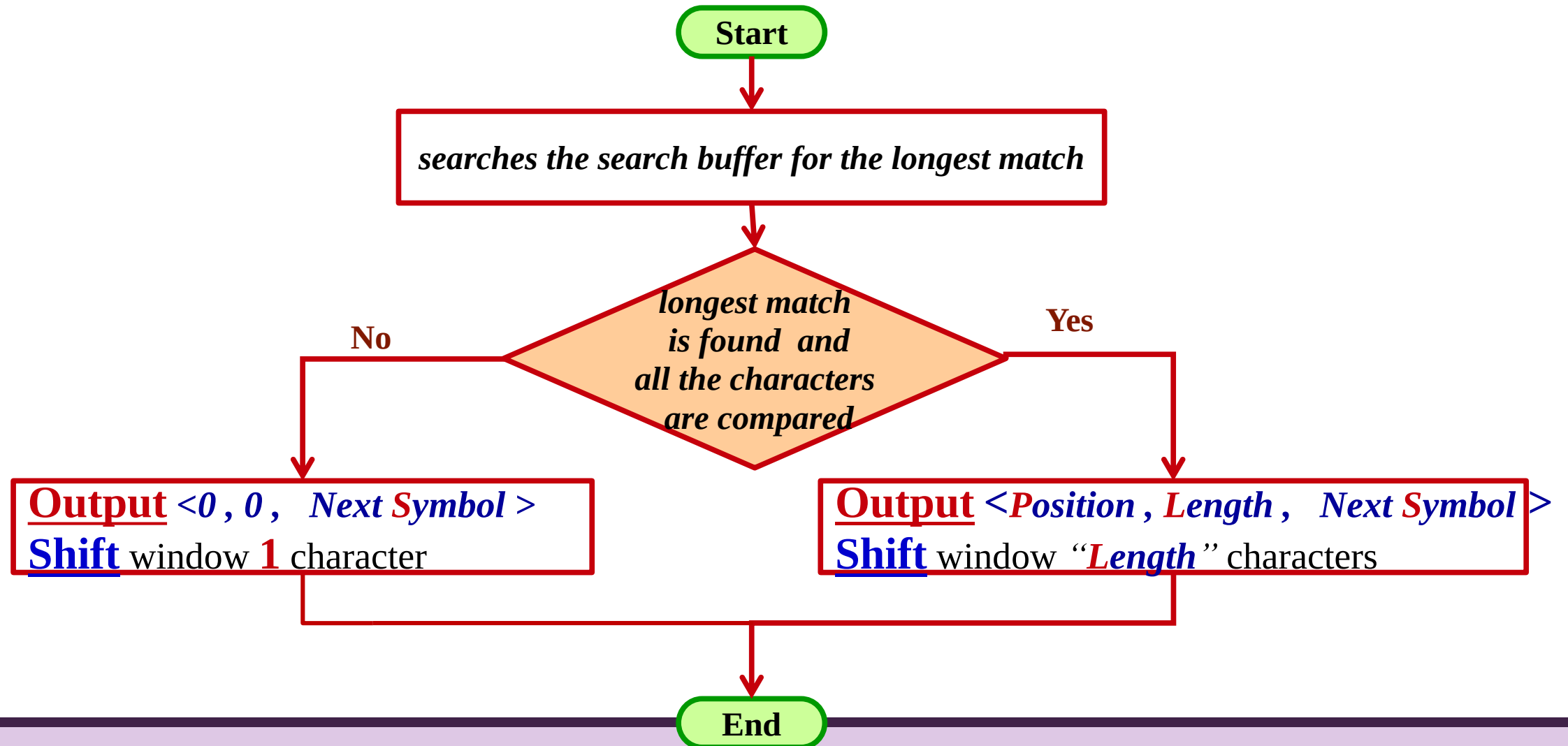
Next Symbol : the codeword corresponding to the symbol in the look-ahead buffer that follows the match

LZ 77 (Compression)

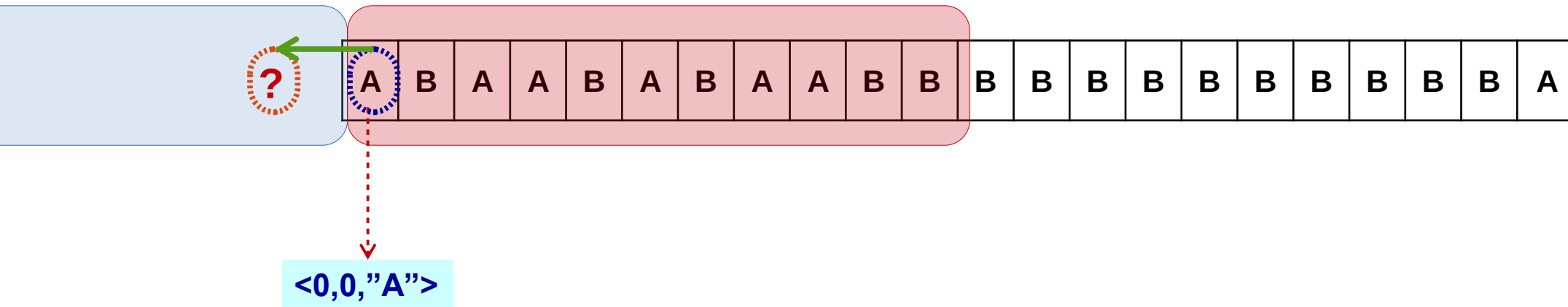


***“BA”** exists in search buffer*
Go Back three Steps, Pick Two Symbol
*Position=**3**, **L**ength =**2**, next **S**ymbol=**“B”***

LZ 77 Algorithm

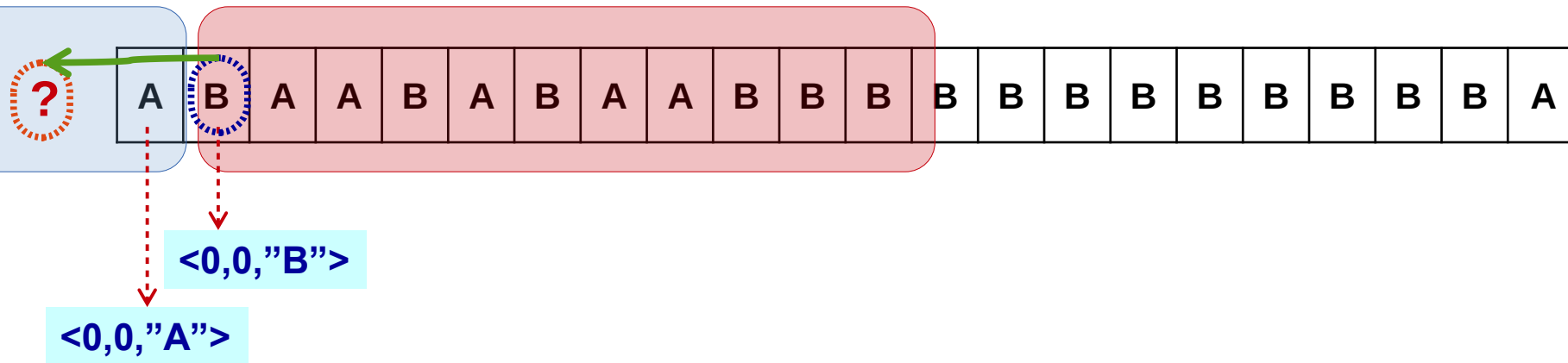


LZ 77 (Compression)



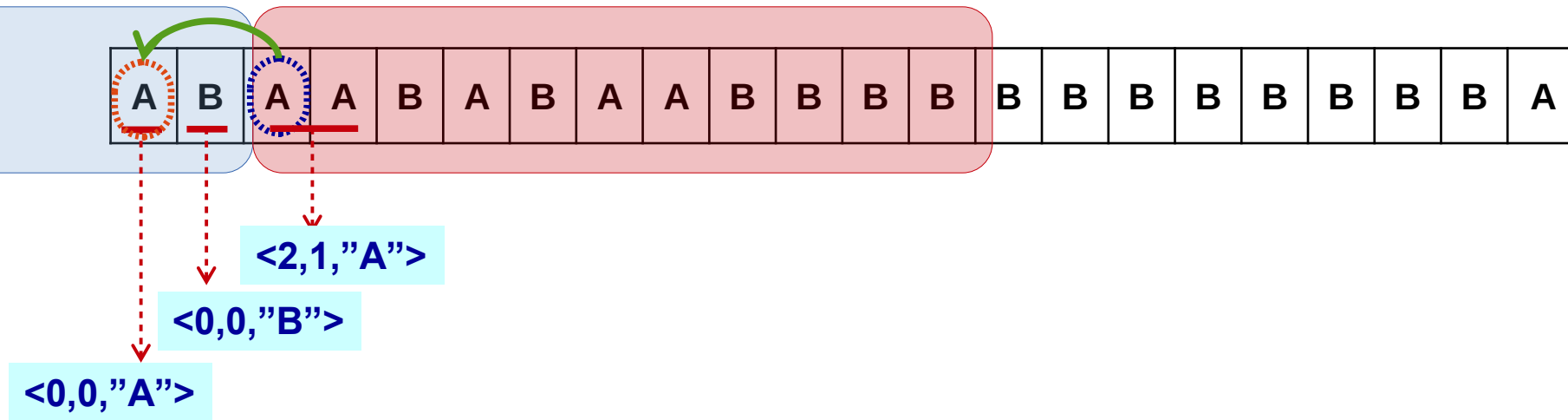
There is no “A” in search buffer
Position=0, Length =0, next Symbol=“A”

LZ 77 (Compression)



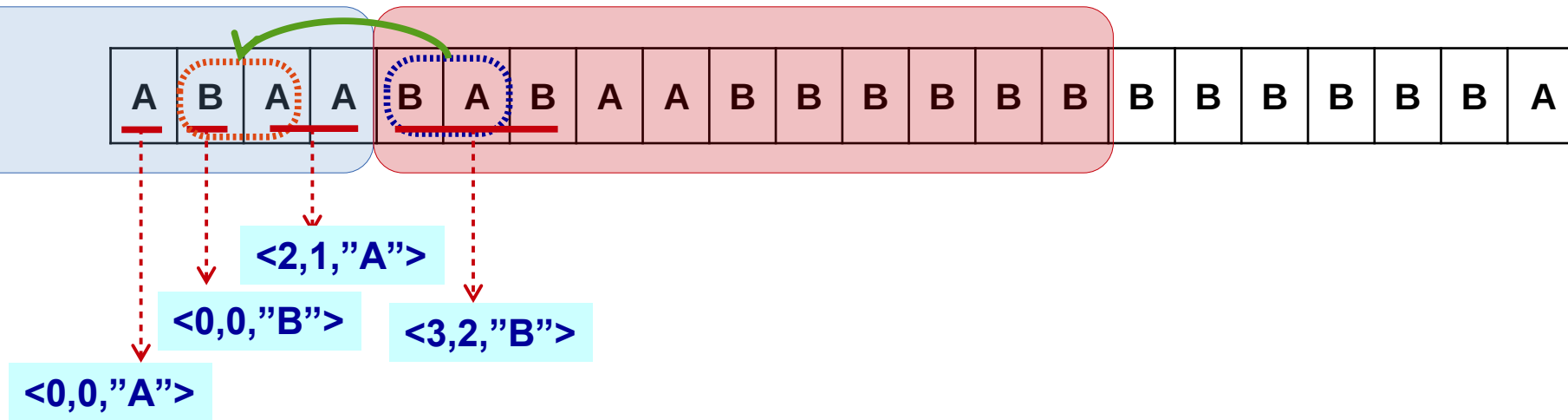
*There is no “**B**” in search buffer*
***P**osition=0, **L**ength =0, next **S**ymbol=“**B**”*

LZ 77 (Compression)



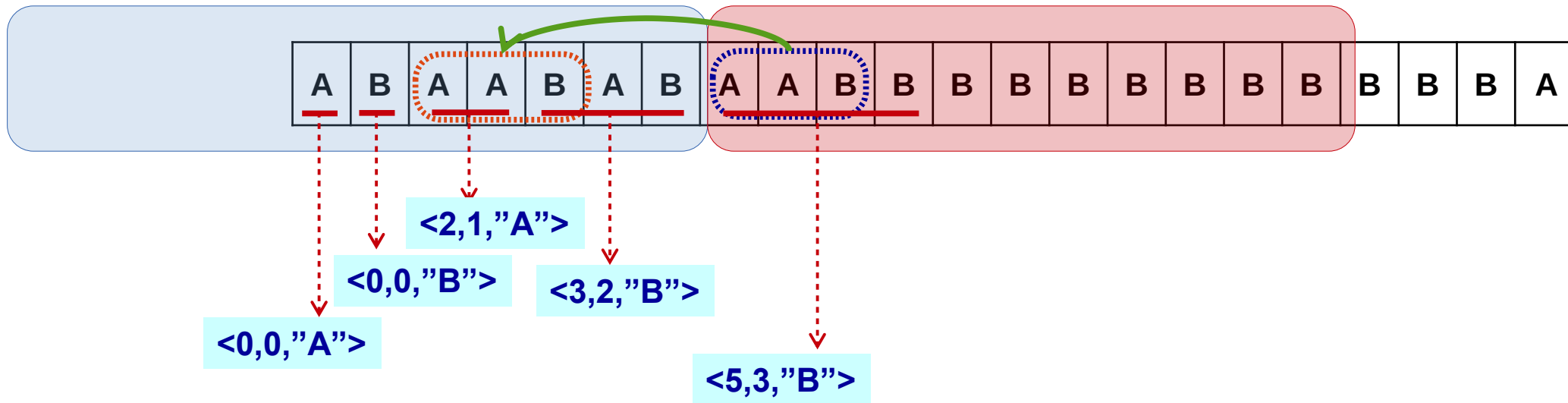
*Only “**A**” exists in search buffer*
Go Back two Steps, Pick One Symbol
P**osition=**2**, **L**ength =**1**, next **S**ymbol=**”A”

LZ 77 (Compression)



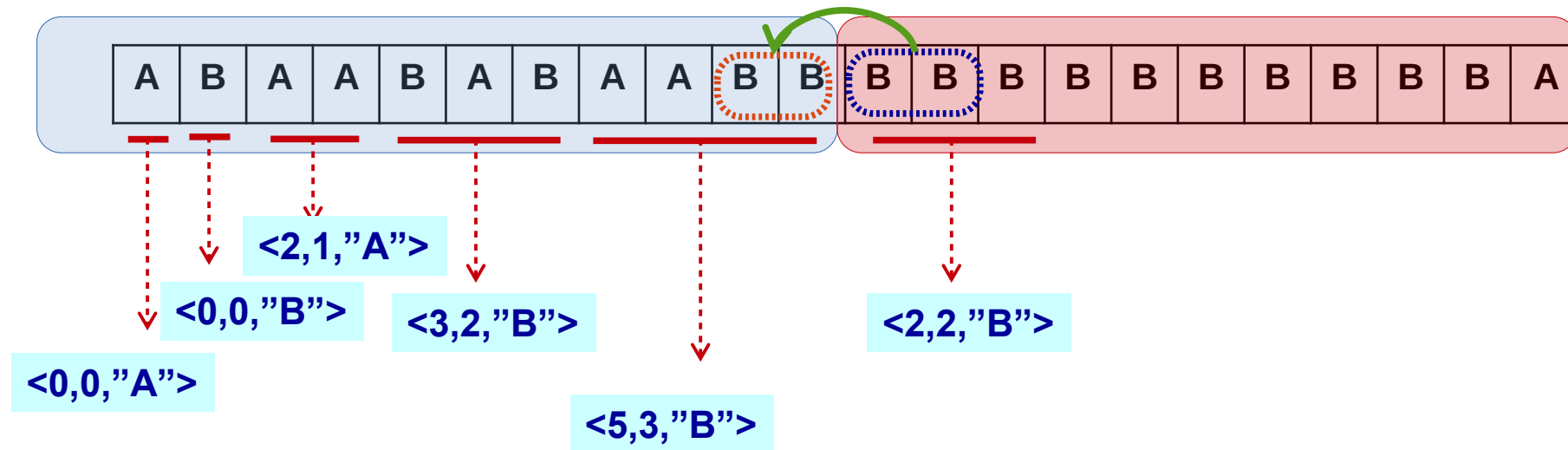
***"BA"** exists in search buffer*
Go Back three Steps, Pick Two Symbol
*Position=**3**, **L**ength =**2**, next **S**ymbol="**B**"*

LZ 77 (Compression)



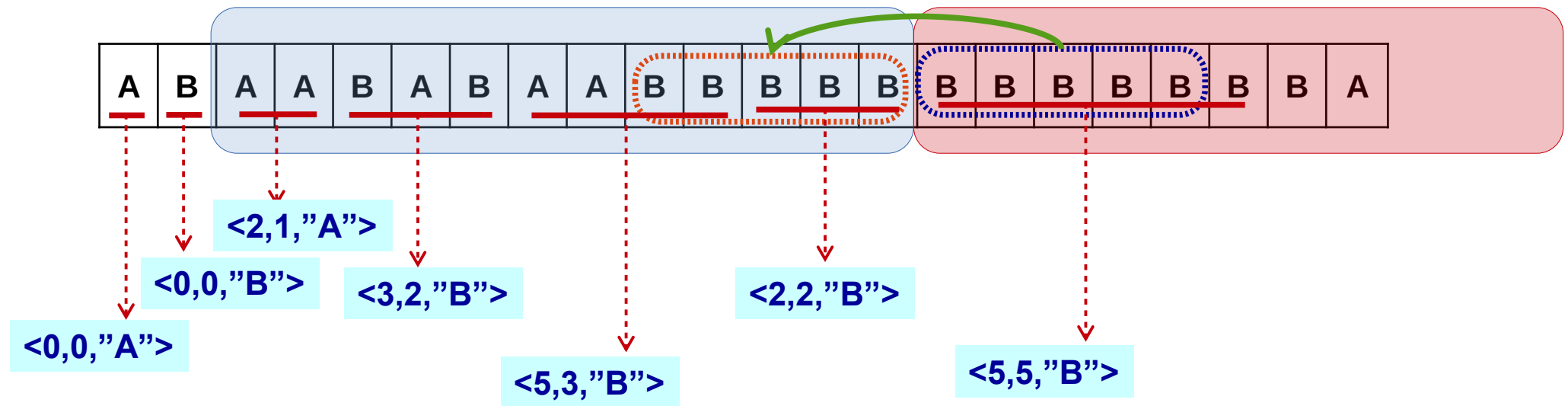
"AAB" exists in search buffer
Go Back five Steps, Pick Three Symbol
Position=5, Length =3, next Symbol="B"

LZ 77 (Compression)



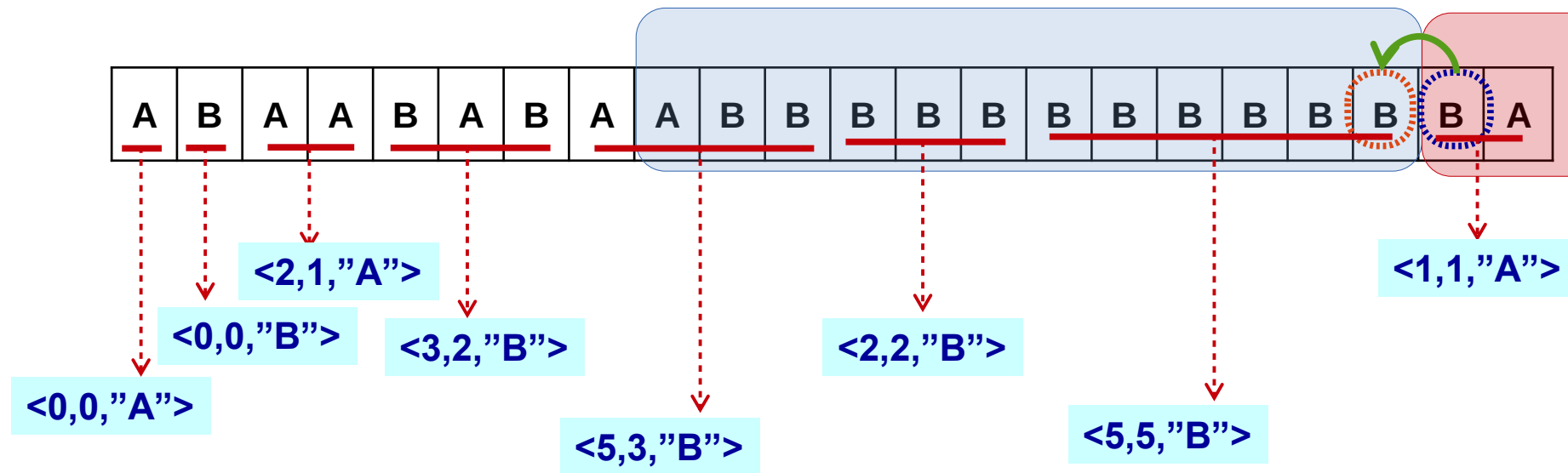
***"BB"** exists in search buffer*
Go Back two Steps, Pick two Symbol
Position=2, Length=2, next Symbol="B"

LZ 77 (Compression)



*“**BBBBB**” exists in search buffer*
Go Back five Steps, Pick five Symbol
*Position=**5**, **L**ength =**5**, next **S**ymbol=**”B”***

LZ 77 (Compression)



***"B"** exists in search buffer*
Go Back One Steps, Pick One Symbol
***Position**=1, **Length** =1, next **Symbol**="A"*

LZ 77 (Compression Ratio)

Remember

1 Bit can represent **2 Values** (0,1) [0-1]

2 Bits can represent **4 values** (00,01,10,11) [0-3]

3 Bits can represent **8 Values** (000,001,010,011,100,101,110,111) [0-7]

In General

N Bits can be used to represent **2^N Values** [0 - ($2^N - 1$)]

LZ 77 (Compression Ratio)

Tag = < Position, Length, Next Symbol Code>

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<5,3,"B">

<2,2,"B">

<5,5,"B">

<1,1,"A">

Original Size = Number of Symbols * Bits used to Store one Symbol
= 22 Symbols * 8 Bits / Symbol = 176 bits
(Store "Symbol" ASCII Code in 8 Bits)

Max "Position" Value = 5

Max "Length" Value = 5

Max Symbols = 256 Symbol

Tag size = 3 + 3 + 8 = 14 Bits

Store "Position" Value in 3 Bits

Store "Length" Value in 3 Bits

Store "Symbol" ASCII Code in 8 Bits

Number of Tags = 8 Tags

Compressed Size = 8*14 = 112 bits

LZ 77 (Compression Ratio)

Tag = < Position, Length ,Next Symbol Code>

Effect of Increasing length of Search Window

Higher Probability to find matched strings (Decrease Number of Tags) 

Increase Number of Bits used to Store “**Position**” values 

Effect of Increasing length of Look Ahead Buffer

Higher Probability to match longer strings(Decrease Number of Tags) 

Increase Number of Bits used to Store “**Length**” values 

LZ 77 (Decompression)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<5,3,"B">

<2,2,"B">

<5,5,"B">

<1,1,"A">

A																				
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Don't pick any symbol from Search Window

*Add **S**ymbol="A"*

LZ 77 (Decompression)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A"> <0,0,"B"> <2,1,"A"> <3,2,"B"> <5,3,"B"> <2,2,"B"> <5,5,"B"> <1,1,"A">

A	B																			
---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Don't pick any symbol from Search Window
*Add **S**ymbol="B"*

LZ 77 (Decompression)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

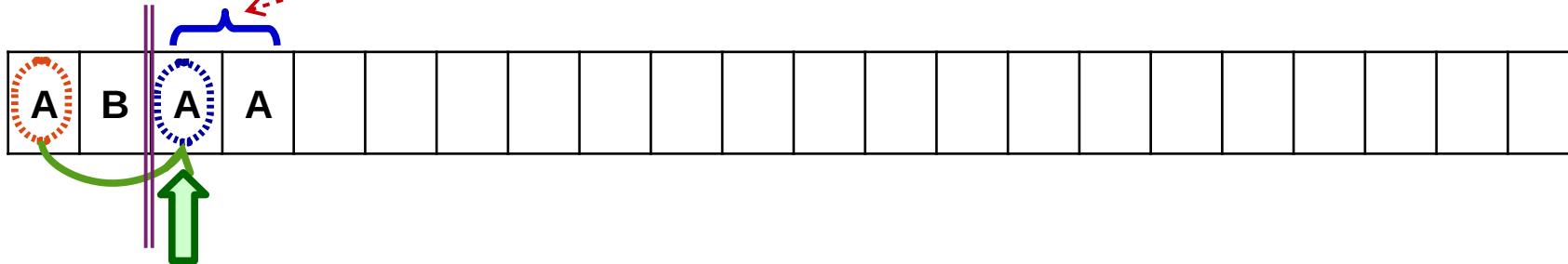
<3,2,"B">

<5,3,"B">

<2,2,"B">

<5,5,"B">

<1,1,"A">



Go back **Two** Positions in search window
pick **One** symbol from Search Window
Add **Symbol**="A"

LZ 77 (Decompression)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

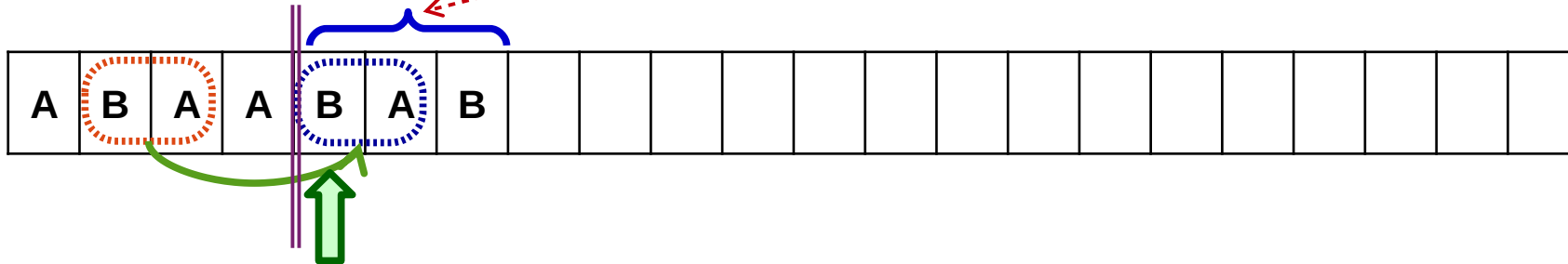
<3,2,"B">

<5,3,"B">

<2,2,"B">

<5,5,"B">

<1,1,"A">



Go back **Three** Positions in search window
pick **Two** symbols from Search Window
Add **Symbol**="B"

LZ 77 (Decompression)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

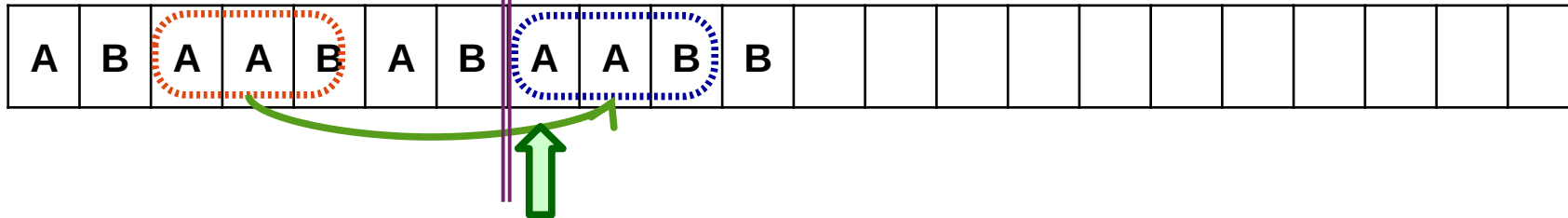
<3,2,"B">

<5,3,"B">

<2,2,"B">

<5,5,"B">

<1,1,"A">



Go back **Five** Positions in search window
pick **Three** symbols from Search Window
Add **Symbol**="B"

LZ 77 (Decompression)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<5,3,"B">

<2,2,"B">

<5,5,"B">

<1,1,"A">

A	B	A	A	B	A	B	A	A	B	B	B	B	B								
---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--	--	--	--	--

Go back **Two** Positions in search window
pick **Two** symbols from Search Window
Add **Symbol**="B"

LZ 77 (Decompression)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

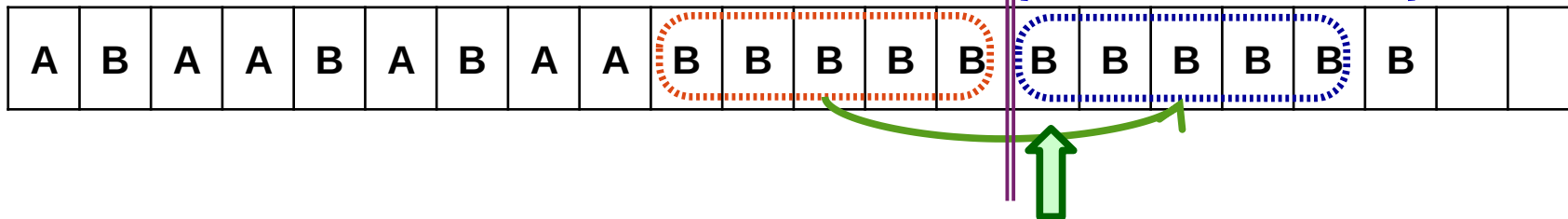
<3,2,"B">

<5,3,"B">

<2,2,"B">

<5,5,"B">

<1,1,"A">



Go back **Five** Positions in search window
pick **Five** symbols from Search Window
Add **Symbol**="B"

LZ 77 (Decompression)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<5,3,"B">

<2,2,"B">

<5,5,"B">

<1,1,"A">

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Go back **One** Positions in search window
pick **One** symbol from Search Window
Add **Symbol**="B"

Example

Apply the following LZ 77 compression techniques on that stream and calculate the ratio of the compressed size w.r.t. the original size.

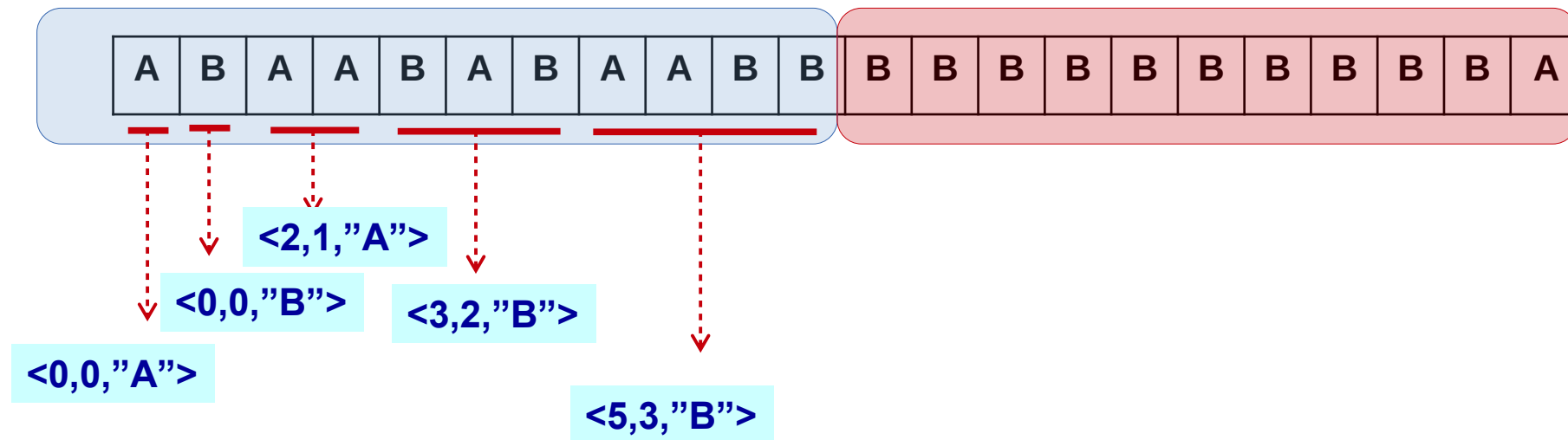
aaaabbababbbaaabbbaaaaaaaaaa

where - Each character is saved in a byte

LZ 77 (Compression) (Handling Repetitive Sequence)

Back to Previous Example

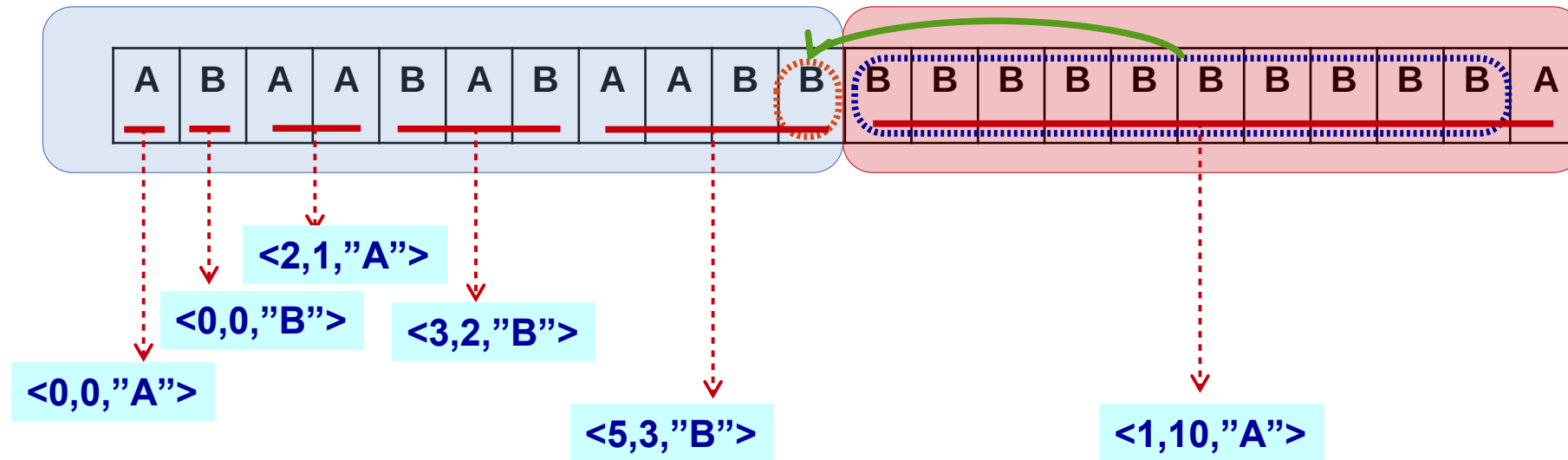
Can We manipulate Consecutive “B”s more efficient ?



*“**AAB**” exists in search buffer*
Go Back five Steps, Pick Three Symbol
*Position=**5**, **L**ength =**3**, next **S**ymbol=**B**”*

LZ 77 (Compression) (Handling Repetitive Sequence)

YES, We Can



*There are **Ten Consecutive** “B” in Look Ahead Buffer
“B” exists in search buffer One position Backward
Go Back **One Steps**, Pick **Ten Symbols**
Position=1, Length =10, next Symbol=“A”*

LZ 77 (Decompression) (Handling Repetitive Sequence)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<5,3,"B">

<1,10,"A">

A	B	A	A	B	A	B	A	A	B	B	B	B								
---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--	--	--	--	--

Pick Three (out of Ten) Symbol

Go back **One** Position in search window
pick **Ten** symbols from Search Window (in 10 Steps)
Add **Symbol**="A"

LZ 77 (Decompression) (Handling Repetitive Sequence)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<5,3,"B">

<1,10,"A">

A	B	A	A	B	A	B	A	A	B	B	B	B	B							
---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--	--	--	--

Pick Four (out of Ten) Symbol

Repeat , Five, Six, Seven, Eight, Nine, and Ten

Go back **One** Position in search window
pick **Ten** symbols from Search Window (in 10 Steps)
Add **Symbol="A"**

LZ 77 (Decompression) (Handling Repetitive Sequence)

Original Data

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<5,3,"B">

<1,10,"A">

A	B	A	A	B	A	B	A	A	B	B	B	B	B	B	B	B	B	B	B	A
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pick Ten (out of Ten) Symbol

Go back **One** Position in search window
pick **Ten** symbols from Search Window (in 10 Steps)
Add **Symbol**="A"

LZ77 (Compression Ratio)

Tag = < Position, Length ,Next Symbol Code>

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<5,3,"B">

<1,10,"A">

Original Size = Number of Symbols * Bits used to Store one Symbol
= 22 Symbols * 8 Bits / Symbol = 176 bits
(Store "Symbol" ASCII Code in 8 Bits)

Max "Position" Value = 5

Max "Length" Value = 10

Max Symbols = 256 Symbol

Tag size = 3 + 4 + 8 = 15 Bits

Store "Position" Value in 3 Bits

Store "Length" Value in 4 Bits

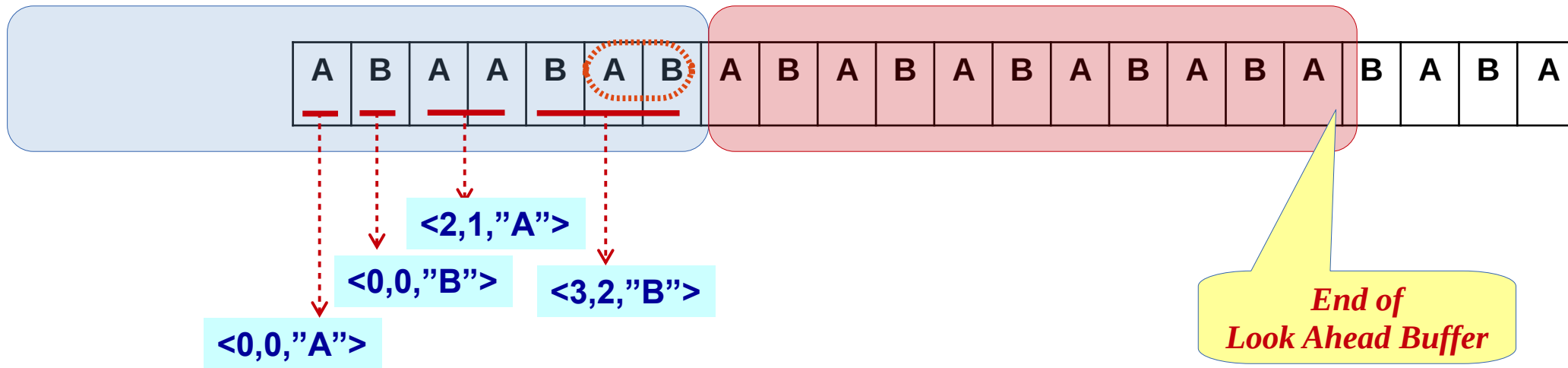
Store "Symbol" ASCII Code in 8 Bits

Number of Tags = 6 Tags

Compressed Size = 6 * 15 = 90 bits

LZ 77 (Compression) (Handling Repetitive Sequence)

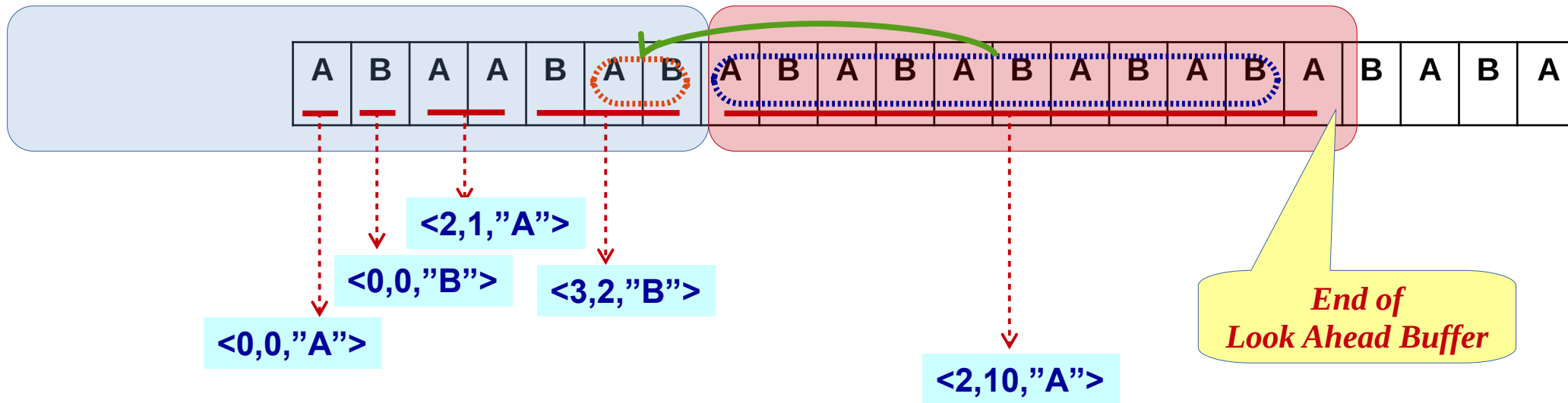
*Can We Apply the same Technique on
Consecutive “Two Symbols” ?*



*There are **Ten** Consecutive Symbols “AB” in Look Ahead Buffer
“AB” exists in search buffer Adjacent to Look Ahead Buffer
Go Back **Two** Steps, Pick **Ten** Symbols
Position=2, **Length =10**, next **Symbol**=“A”*

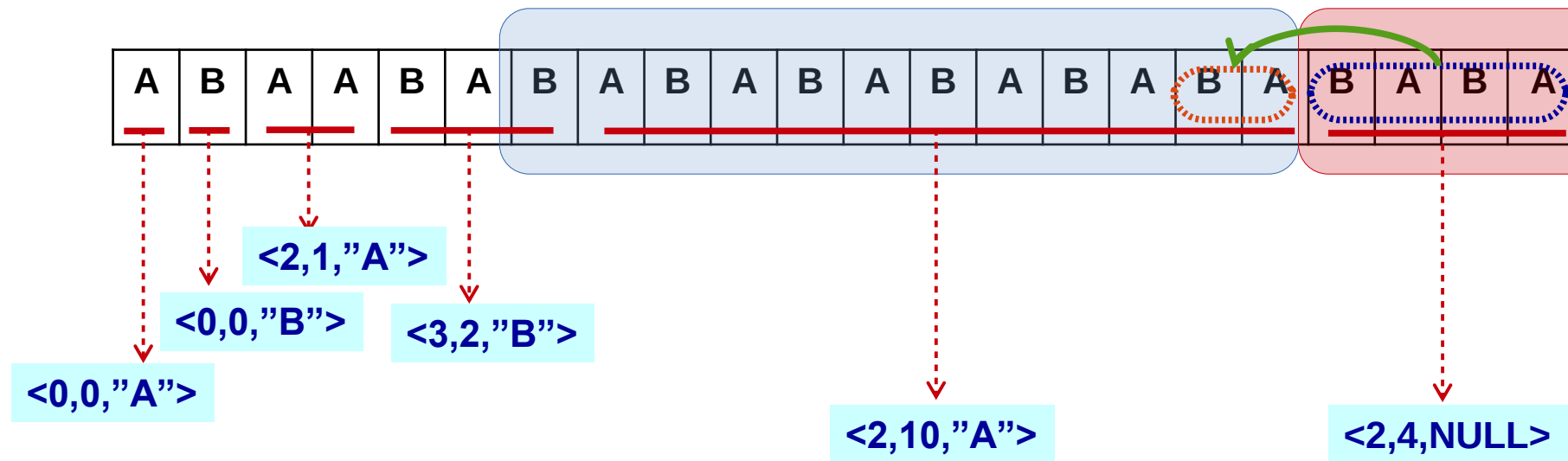
LZ 77 (Compression) (Handling Repetitive Sequence)

*Can We Apply the same Technique on
Consecutive “Two Symbols” ?*



*There are **Ten Consecutive Symbols** “AB” in Look Ahead Buffer
“AB” exists in search buffer Adjacent to Look Ahead Buffer
Go Back **Two Steps**, Pick **Ten Symbols**
Position=2, Length =10, next Symbol=“A”*

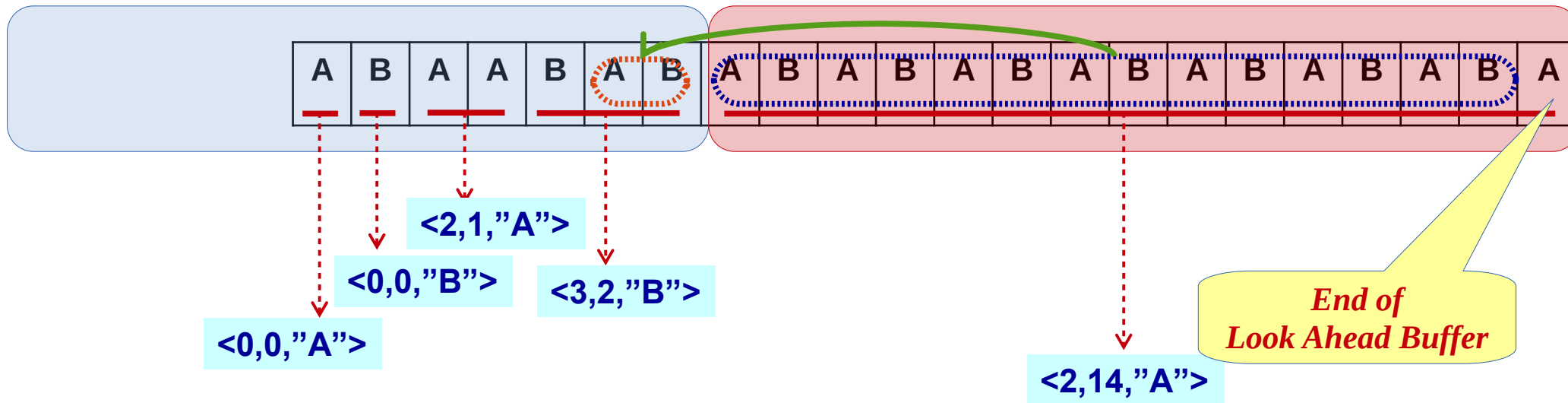
LZ 77 (Compression) (Handling Repetitive Sequence)



There are **Four Consecutive Symbols** “BA” in Look Ahead Buffer
“BA” exists in search buffer Adjacent to Look Ahead Buffer
Go Back **Two Steps**, Pick **Four Symbols**
Position=2, Length =4, next Symbol=NULL

LZ 77

What if we use BIGGER look Ahead Buffer ?



There are **14 Consecutive Symbols "AB"** in Look Ahead Buffer
"AB" exists in search buffer Adjacent to Look Ahead Buffer
Go Back **Two Steps**, Pick **Ten Symbols**
Position=2, Length =14, next Symbol="A"

LZ77 (Compression Ratio)

Tag = < Position, Length, Next Symbol Code>

<0,0,"A">

<0,0,"B">

<2,1,"A">

<3,2,"B">

<2,14,"A">

Original Size = Number of Symbols * Bits used to Store one Symbol
= 22 Symbols * 8 Bits / Symbol = 176 bits
(Store "Symbol" ASCII Code in 8 Bits)

Max "Position" Value = 3

Max "Length" Value=14

Max Symbols = 256 Symbol

Tag size = 2 + 4 + 8 =14 Bits

Store "Position" Value in 2 Bits

Store "Length" Value in 4 Bits

Store "Symbol" ASCII Code in 8 Bits

Number of Tags = 5 Tags

Compressed Size=5*14=70 bits

LZ 77 (Compression)

(Handling Repetitive Overlapped Sequence)

C A B R A C A D A B R A R R A R R A D

<0,0,"C">

C A B R A C A D A B R A R R A R R A D

<0,0,"A">

C A B R A C A D A B R A R R A R R A D

<0,0,"B">

C A B R A C A D A B R A R R A R R A D

<0,0,"R">

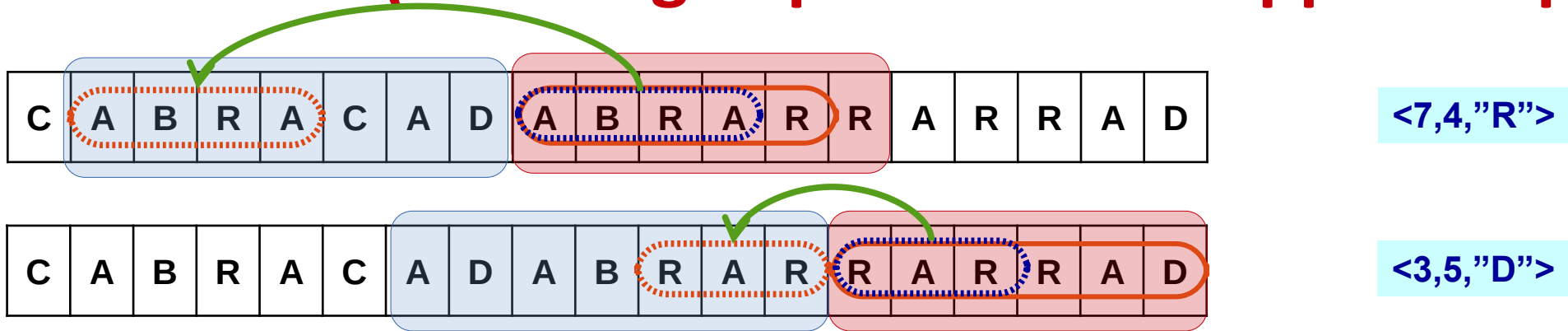
C A B R A C A D A B R A R R A R R A D

<3,1,"C">

C A B R A C A D A B R A R R A R R A D

<2,1,"D">

LZ 77 (Compression) (Handling Repetitive Overlapped Sequence)



Original Size = Number of Symbols * Bits used to Store one Symbol
= 19 Symbols * 8 Bits / Symbol = 152 bits
(Store "Symbol" ASCII Code in 8 Bits)

Max "Position" Value = 7
Max "Length" Value = 5
Max Symbols = 256 Symbol
Tag size = 3 + 3 + 8 = 14 Bits

Store "Position" Value in 3 Bits
Store "Length" Value in 3 Bits
Store "Symbol" ASCII Code in 8 Bits

Number of Tags = 8 Tags
Compressed Size = 8 * 14 = 112 bits

Advantages and Disadvantage of LZ77

Advantages of LZ77

- Probabilities of symbols is not required to be known a priori.
- That is, the longer the size of the sliding window, the better the performance of data compression
- No coding table Required for Decompression.

Disadvantage of LZ77

A straightforward implementation would require up to [Look Ahead Buffer Size] * [Search Window Size] Symbol comparisons per Tag produced.